Геометрия

Плотников Даниил Михайлович

Санкт-Петербургский государственный университет

Оглавление

Точки и векторы 5
Основные понятия 6
Программное представление точки 7
Расстояние между точками . 9
Операции с векторами 10
Программное представление вектора 12
Угол между векторами 13
Прямые 14
Определение 15
Программное представление 16
Параллельность прямых 17
Пересечение прямых 18
Угол между прямыми 19
Задача: Точка в прямоугольнике 20
Положение точки относительно прямой 21
Зависимость знака от нормального вектора 22

Алгоритм Коэна-Сазерленда 23
Выбор векторов 24
Ориентация точек 26
Подсчёт точек 28
Расстояние и проекция точки на прямую 30
Отражение точки относительно прямой 31
Отрезки 32
Операции с отрезками 34
Пересечение отрезков 35
Многоугольники 36
Определение 37
Периметр многоугольника . 38
Площадь многоугольника . . 40
Доказательство 41
Центроид (центр масс) 43
Доказательство 44
Выпуклость 46
Проверка простоты многоугольника 48
Точка внутри многоугольника 49
Метод лучей (Ray Casting) . . 50
Код метода лучей (Ray Casting) 51

Определение намотки (Winding Number) 52
Метод ненулевой намотки (Winding Number) 53
Триангуляция многоугольника 55
Алгоритм отсечения ушей (Ear Clipping) 56
Выпуклая оболочка . . . 60
Определение 61
Свойства выпуклой оболочки 62
Алгоритм Грэхема 63

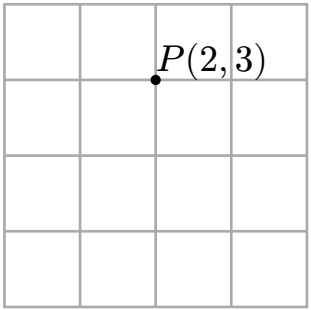
Точки и векторы

Основные понятия

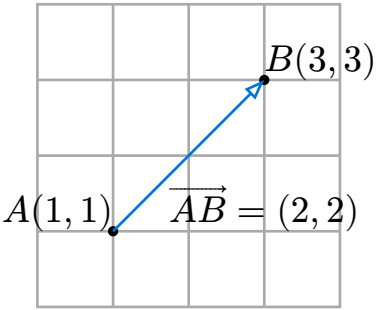
Точка — пара вещественных чисел (x, y), представляющая положение в плоскости.

Вектор — направленный отрезок от точки A к точке B, обозначается $\overrightarrow{AB}$ или (dx, dy), где $dx = B_x - A_x$, $dy = B_y - A_y$.

Точка



Вектор



Программное представление точки

```
struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    Point operator+(const Vector& v) const { return Point(x + v.x, y +
v.y); }
    Point operator-(const Vector& v) const { return Point(x - v.x, y -
v.y); }
    bool operator==(const Point& p) const {
      return fabs(x - p.x) < EPS && fabs(y - p.y) < EPS;
    }
};

double dist(const Point& a, const Point& b) {
    return hypot(a.x - b.x, a.y - b.y);
}
```

В геометрических вычислениях избегайте прямого сравнения вещественных чисел с плавающей точкой. Используйте сравнение с epsilon: `fabs(a - b) < EPS`.

Расстояние между точками

Расстояние между двумя точками A и B — длина отрезка, соединяющего их.

Формула следует из теоремы Пифагора: расстояние – гипотенуза прямоугольного треугольника с катетами Δx и Δy.

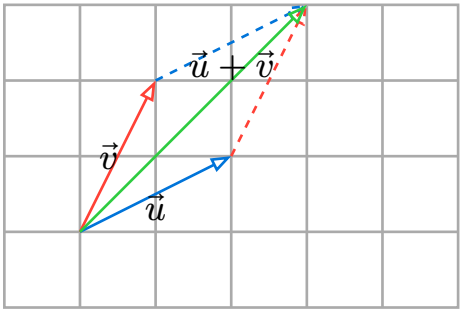
`hypot(a, b)` вычисляет $\sqrt{a^2 + b^2}$ с повышенной точностью, избегая переполнения промежуточных значений. Возвращает `double`.

Для большей точности используйте `hypotl` с `long double`.

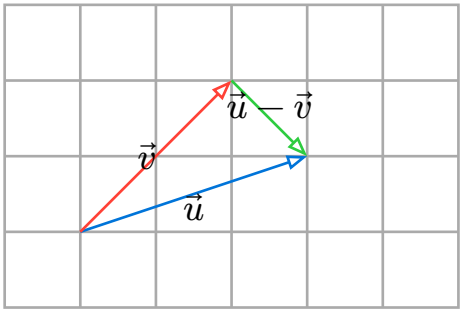
Операции с векторами

Пусть $\vec{u} = (u_x, u_y)$, $\vec{v} = (v_x, v_y)$. Рассмотрим операции с векторами.

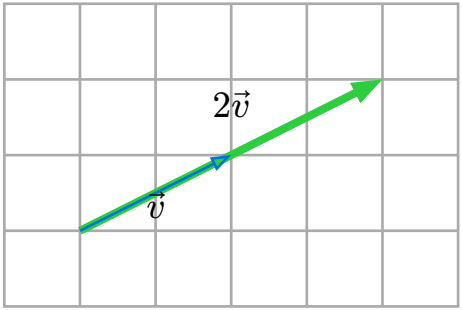
Сложение: $\vec{u} + \vec{v} = (u_x + v_x, u_y + v_y)$



Вычитание: $\vec{u} - \vec{v} = (u_x - v_x, u_y - v_y)$



Скаляр на вектор: $k\vec{v} = \left(kv_x, kv_y\right)$



- Длина вектора: $|\vec{v}| = \sqrt{v_x^2 + v_y^2}$
- Скалярное произведение: $\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y = |u|\ |v| \cos\theta$
- Векторное произведение: $\vec{u} \times \vec{v} = u_x v_y - u_y v_x = |u|\ |v| \sin\theta$

Вообще говоря, формально векторное произведение определяется не так. Оно определено как вектор такой же длины, но перпендикулярный обоим исходным векторам.

Программное представление вектора

```
struct Vector {
    double x, y;
    Vector(double x = 0, double y = 0) : x(x), y(y) {}
    Vector operator+(const Vector& v) const { return Vector(x + v.x, y +
v.y); }
    Vector operator-(const Vector& v) const { return Vector(x - v.x, y -
v.y); }
    Vector operator*(double k) const { return Vector(x * k, y * k); }
    bool operator==(const Vector& v) const {
      return fabs(x - v.x) < EPS && fabs(y - v.y) < EPS;
    }
};

double dot(const Vector& a, const Vector& b) { return a.x*b.x + a.y*b.y; }
double cross(const Vector& a, const Vector& b) { return a.x*b.y - a.y*b.x; }
double length(const Vector& v) { return hypot(v.x, v.y); }
```

Угол между векторами

Угол θ между векторами $\vec{u}$ и $\vec{v}$:

$$\cos\theta = \frac{\vec{u}\cdot\vec{v}}{|\vec{u}|\ |\vec{v}|}$$

Формула следует из определения скалярного произведения: $\vec{u}\cdot\vec{v} = |u|\ |v|\cos\theta$.

```
double angle(const Point& a, const Point& b) {
    return atan2(b.y - a.y, b.x - a.x);
}
```

`atan2(y, x)` возвращает угол в радианах от оси OX к вектору (x, y), в диапазоне $(-\pi, \pi]$. Обрабатывает все квадранты правильно. Возвращает `double`.

Для большей точности используйте `atan2l` с `long double`.

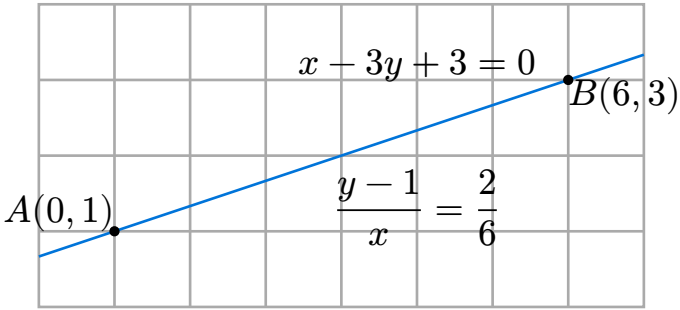
Прямые

Определение

Прямая — это алгебраическая линия первого порядка, множество точек на плоскости (или в пространстве), координаты которых удовлетворяют линейному уравнению первой степени.

Прямая может быть задана уравнением $ax + by + c = 0$.

Альтернативно, через две точки A и B: $\dfrac{y - A_y}{x - A_x} = \dfrac{B_y - A_y}{B_x - A_x}$



Программное представление

```
struct Line {
    double a, b, c;
    Line(double a = 0, double b = 0, double c = 0) : a(a), b(b), c(c) {}
    Line(const Point& p1, const Point& p2) {
        a = p2.y - p1.y;
        b = p1.x - p2.x;
        c = p1.y * p2.x - p1.x * p2.y;
    }
};
```

Формула коэффициентов уравнения прямой по двум точкам:

$$\begin{cases} a \cdot \mathrm{p1}.x + b \cdot \mathrm{p1}.y + c = 0 \\ a \cdot \mathrm{p2}.x + b \cdot \mathrm{p2}.y + c = 0 \end{cases} \Rightarrow a(\mathrm{p2}.x - \mathrm{p1}.x) + b(\mathrm{p2}.y - \mathrm{p1}.y) = 0$$

Поскольку уравнение однородное относительно a и b, выбираем $a = \mathrm{p2}.y - \mathrm{p1}.y$, $b = \mathrm{p1}.x - \mathrm{p2}.x$

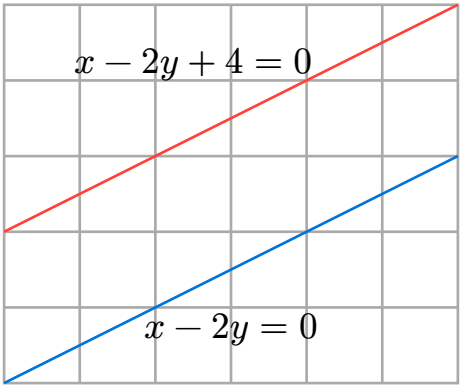
Тогда $c = -a \cdot \mathrm{p1}.x - b \cdot \mathrm{p1}.y = \mathrm{p1}.y \cdot \mathrm{p2}.x - \mathrm{p1}.x \cdot \mathrm{p2}.y$

Параллельность прямых

Две прямые параллельны, если их нормальные векторы коллинеарны:

$a_1 b_2 - a_2 b_1 = 0.$

```
bool parallel(const Line& l1, const Line& l2) {
    return fabs(l1.a * l2.b - l1.b * l2.a) < EPS;
}
```



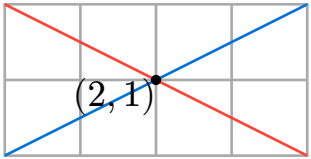
Пересечение прямых

Точка пересечения прямых находится решением системы:

$$\begin{cases} a_1x + b_1y + c_1 = 0 \\ a_2x + b_2y + c_2 = 0 \end{cases}$$

Формула: $x = \dfrac{b_1c_2 - b_2c_1}{a_1b_2 - a_2b_1}, y = \dfrac{a_2c_1 - a_1c_2}{a_1b_2 - a_2b_1}$

```
Point intersection(const Line& l1, const Line& l2) {
    double d = l1.a * l2.b - l1.b * l2.a;
    double dx = l1.b * l2.c - l1.c * l2.b;
    double dy = l1.c * l2.a - l1.a * l2.c;
    return Point(dx / d, dy / d);
}
```

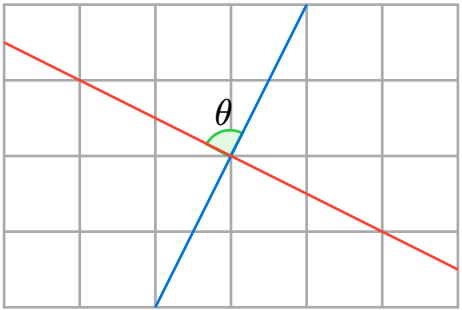


Угол между прямыми

Угол θ между двумя прямыми: $\tan\theta = |\dfrac{a_1 b_2 - a_2 b_1}{a_1 a_2 + b_1 b_2}|$

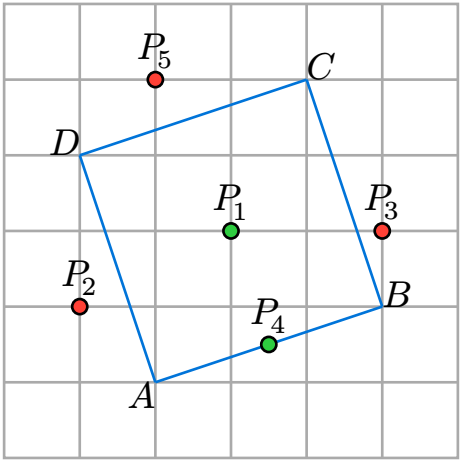
Используется формула тангенса разности углов наклона.

```
double angleBetweenLines(const Line& l1, const Line& l2) {
    double denom = l1.a * l2.a + l1.b * l2.b;
    double num = l1.a * l2.b - l2.a * l1.b;
    return fabs(atan2(num, denom));
}
```



Задача: Точка в прямоугольнике

Дан прямоугольник с вершинами A, B, C, D. Посчитать сколько точек из массива P находятся внутри прямоугольника. Будем считать что точка лежащая на стороне находится внутри прямоугольника.

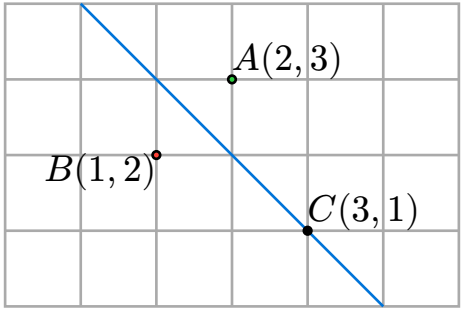


Положение точки относительно прямой

Для точки (A_x, A_y) и прямой $ax + by + c = 0$:

- Если $aA_x + bA_y + c > 0$, точка находится выше прямой (для стандартной ориентации).
- Если $aA_x + bA_y + c < 0$, точка находится ниже прямой.
- Если $aA_x + bA_y + c = 0$, точка лежит на прямой.

$$x + y - 4 = 0$$



$A : 2 + 3 - 4 = 1$

$B : 1 + 2 - 4 = -1$

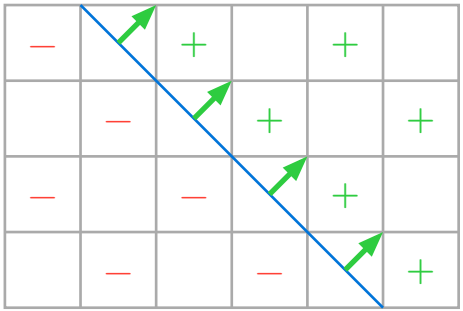
$C : 3 + 1 - 4 = 0$

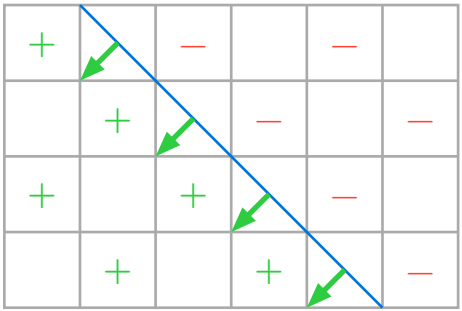
Зависимость знака от нормального вектора

Знак выражения $ax + by + c$ зависит от направления нормального вектора (a, b).

Если поменять направление нормального вектора на противоположное $(-a, -b)$, то и знак изменится на противоположный.

Поэтому важно заранее определить, какая сторона прямой считается “положительной”.





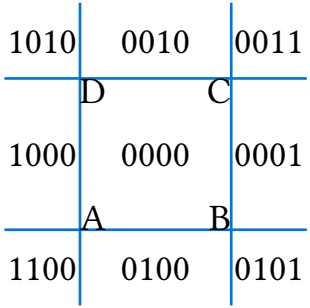
Алгоритм Коэна-Сазерленда

Не особо нужный в рамках спортивного программирования, но полезный в компьютерной графике алгоритм для отсечения отрезков использует кодирование, позволяющее легко понять как расположены объекты внутри прямоугольника.

Для быстрой проверки принадлежности точки прямоугольнику используется 4-битное кодирование:

- Бит 0: слева от левой стороны
- Бит 1: справа от правой стороны
- Бит 2: ниже нижней стороны
- Бит 3: выше верхней стороны

Точка внутри, если код = 0



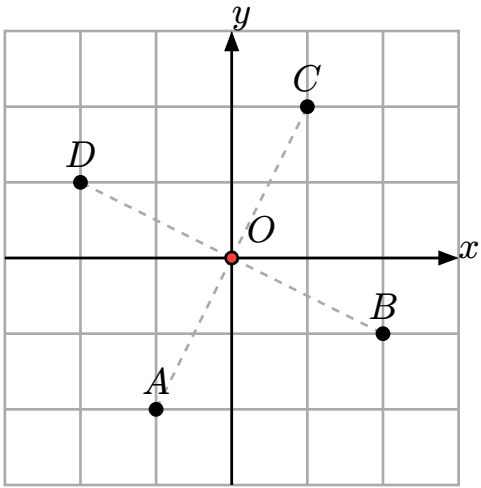
Выбор векторов

Для удобства проверки принадлежности точки многоугольнику, упорядочим вершины так, чтобы внутренняя сторона была с положительной стороны каждого ребра.

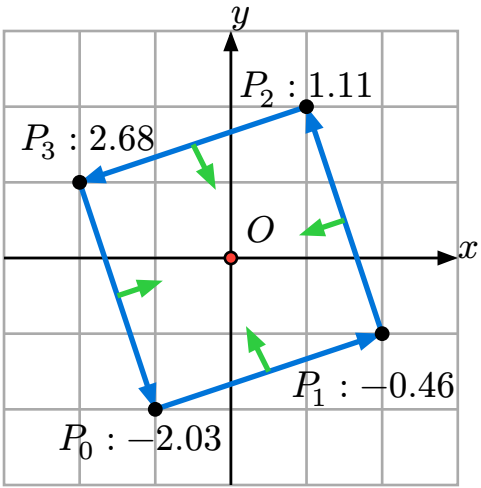
Алгоритм:

1. Найти центр прямоугольника (среднее всех вершин)
2. Отсортировать вершины по полярному углу относительно центра
3. При обходе против часовой стрелки внутренность будет слева (положительная сторона)

До сортировки



После сортировки по углу



При сортировке по углу против часовой стрелки, векторное произведение $(P_{i+1} - P_i) \times (Q - P_i) > 0$ для точек Q внутри многоугольника.

Ориентация точек

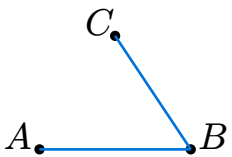
Ориентация трех точек A, B, C – направление поворота от $\overrightarrow{AB}$ к $\overrightarrow{BC}$.

- Положительная: против часовой стрелки (CCW)
- Отрицательная: по часовой стрелке (CW)
- Нулевая: коллинеарны

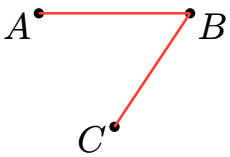
Вычисляется через векторное произведение: $(B - A) \times (C - B)$

```
int orientation(const Point& a, const Point& b, const Point& c) {
    double val = cross(b - a, c - b);
    if (fabs(val) < EPS) return 0;
    return val > 0 ? 1 : 2;
}
```

Положительная CCW



Отрицательная CW



Коллинеарны

```
vector<Point> createRectangle(const Point& A, const Point& B, const Point&
C, const Point& D) {
    vector<Point> pts = {A, B, C, D};
    Point center = {0, 0};
    for (auto& p : pts) {
        center.x += p.x;
        center.y += p.y;
    }
    center.x /= 4;
    center.y /= 4;

    sort(pts.begin(), pts.end(), [&](const Point& p1, const Point& p2) {
        double ang1 = atan2(p1.y - center.y, p1.x - center.x);
        double ang2 = atan2(p2.y - center.y, p2.x - center.x);
        return ang1 < ang2;
    });
    return pts;
}
```

Подсчёт точек

Остаётся только посчитать сколько точек находятся с положительной стороны относительно всех сторон подставляя значения. Это шаг без дополнительной теоретической базы :)

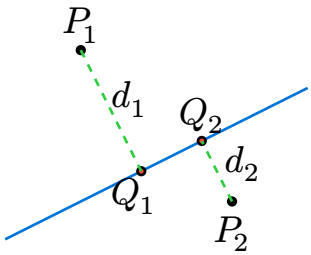
```
bool pointInPolygon(const Point& p, const vector<Point>& poly) {
    for (int i = 0; i < poly.size(); ++i) {
        Point a = poly[i], b = poly[(i+1) % poly.size()];
        if (cross(b - a, p - a) < -EPS) return false;
    }
    return true;
}

int countPointsInside(const vector<Point>& P, const vector<Point>& rect) {
    int count = 0;
    for (const auto& p : P) {
        if (pointInPolygon(p, rect)) {
            count++;
        }
    }
    return count;
}
```

Расстояние и проекция точки на прямую

Ближайшая точка на прямой (проекция) — точка пересечения прямой с перпендикуляром, опущенным из P на прямую.

Расстояние от точки P до прямой $ax + by + c = 0$:

$$d = \frac{|aP_x + bP_y + c|}{\sqrt{a^2 + b^2}}$$



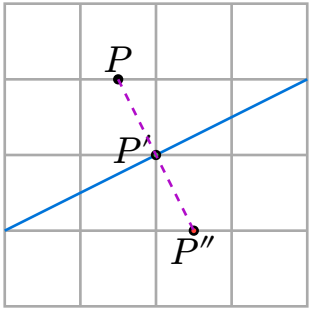

```
double distPointToLine(const Point& p, const Line& l) {
    return fabs(l.a * p.x + l.b * p.y + l.c) / hypot(l.a, l.b);
}
Point closestPointOnLine(const Point& p, const Line& l) {
    double denom = l.a * l.a + l.b * l.b;
    double x = (l.b * (l.b * p.x - l.a * p.y) - l.a * l.c) / denom;
    double y = (l.a * (l.a * p.y - l.b * p.x) - l.b * l.c) / denom;
    return Point(x, y);
}
```

Отражение точки относительно прямой

Отражение точки P относительно прямой L — единственная точка P'' такая, что L является серединным перпендикуляром отрезка PP''.

Вычисляется через проекцию: $P'' = 2P' - P$, где P' – проекция P на L.

```
Point reflectPointOverLine(const Point& p, const Line& l) {
    Point proj = closestPointOnLine(p, l);
    return Point(2 * proj.x - p.x, 2 * proj.y - p.y);
}
```

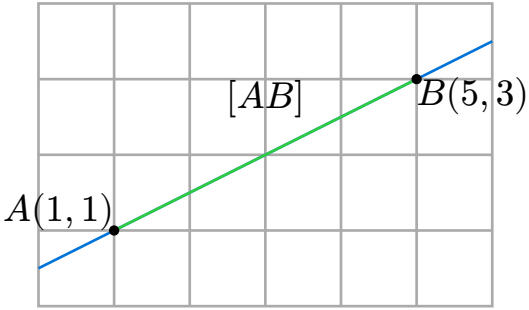


Отрезки

Определение

Отрезок – это часть прямой, ограниченная двумя точками A и B.

```
struct Segment {
    Point a, b;
    Segment(Point a = {0, 0}, Point b = {0, 0}) : a(a), b(b) {}
};
```



Операции с отрезками

Для отрезка $[AB]$:

- Длина: $\sqrt{(B_x - A_x)^2 + (B_y - A_y)^2}$
- Деление в отношении $m : n$: $\left(\dfrac{nA_x + mB_x}{m+n}, \dfrac{nA_y + mB_y}{m+n}\right)$

```
double segmentLength(const Point& a, const Point& b) {
    return hypot(b.x - a.x, b.y - a.y);
}

Point segmentMidpoint(const Point& a, const Point& b) {
    return Point((a.x + b.x) / 2, (a.y + b.y) / 2);
}

Point divideSegment(const Point& a, const Point& b, double m, double n) {
    return Point((n*a.x + m*b.x) / (m + n), (n*a.y + m*b.y) / (m + n));
}
```

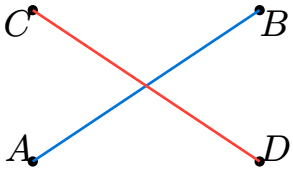
Пересечение отрезков

Два отрезка $[AB]$ и $[CD]$ пересекаются, если:

1. Ориентации A, B, C и A, B, D различны
2. Ориентации C, D, A и C, D, B различны

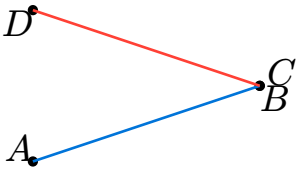
Исключение: если все четыре точки коллинеарны, нужно проверить перекрытие.

Пересекающиеся



Параллельные

Касание в конце



Коллинеарные (перекрываются)

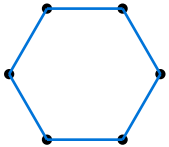
Многоугольники

Определение

Многоугольник – это замкнутая ломаная линия, состоящая из последовательности отрезков, соединяющих вершины. Вершины упорядочены, и первая соединяется с последней.

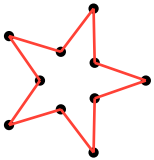
- **Простой многоугольник**: не пересекает сам себя.
- **Выпуклый многоугольник**: все внутренние углы <= 180°, и любая линия между двумя вершинами лежит внутри.
- **Вогнутый многоугольник**: имеет хотя бы один внутренний угол > 180°.

Выпуклый



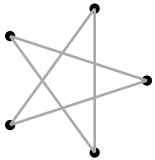
Гексагон

Вогнутый



Звезда

Самопересекающийся

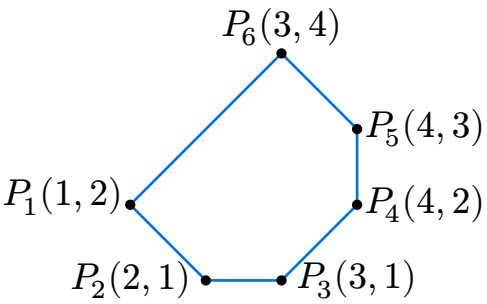


Пентаграмма

Периметр многоугольника

Периметр многоугольника – это сумма длин всех его сторон. Для многоугольника с вершинами $P_1, P_2, ..., P_n$:

$P = \Sigma_{i=1}^{n} \operatorname{dist}(P_i, P_{i+1})$, где $P_{n+1} = P_1$, а dist — расстояние между точками.



Вычисление периметра:

$\operatorname{dist}(P_1, P_2) = \sqrt{1+1} = \sqrt{2} \approx 1.41$

$\operatorname{dist}(P_2, P_3) = \sqrt{1+0} = 1$

$\operatorname{dist}(P_3, P_4) = \sqrt{1+1} = \sqrt{2} \approx 1.41$

$\operatorname{dist}(P_4, P_5) = \sqrt{0+1} = 1$

$\operatorname{dist}(P_5, P_6) = \sqrt{1+1} = \sqrt{2} \approx 1.41$

$\operatorname{dist}(P_6, P_1) = \sqrt{4+4} = \sqrt{8} \approx 2.83$

Сумма $\approx 1.41 + 1 + 1.41 + 1 + 1.41 + 2.83 \approx 8.66$

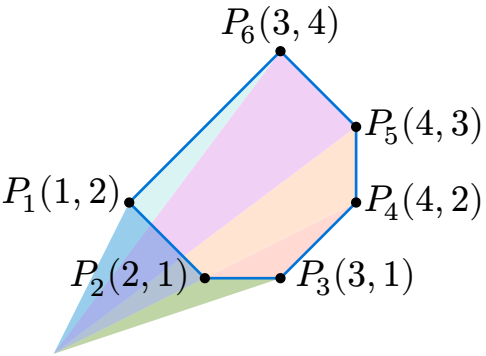
```
double polygonPerimeter(const vector<Point>& p) {
    double perim = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        perim += dist(p[i], p[j]);
    }
    return perim;
}
```

Площадь многоугольника

Для многоугольника с вершинами $P_1, P_2, ..., P_n$ (в порядке обхода по часовой стрелке или против):

$$S = \frac{1}{2}\ |\Sigma_{i=1}^{n}(x_i y_{i+1} - x_{i+1} y_i)|$$

Формула площади Гауса (shoelace/плетенка) геометрически соответствует сумме площадей ориентированных треугольников от начала координат.



Вычисление площади по формуле шнуровки:

$P_1(1,2), P_2(2,1), P_3(3,1), P_4(4,2), P_5(4,3), P_6(3,4)$

$\Sigma = (1 \cdot 1 - 2 \cdot 2) + (2 \cdot 1 - 3 \cdot 1) + (3 \cdot 2 - 4 \cdot 1) + (4 \cdot 3 - 4 \cdot 2) + (4 \cdot 4 - 3 \cdot 3) + (3 \cdot 2 - 1 \cdot 4) = (1 - 4) + (2 - 3) + (6 - 4) + (12 - 8) + (16 - 9) + (6 - 4) = 11 \Rightarrow S = \frac{1}{2}\ |11| = 5.5$

Доказательство

Площадь многоугольника можно разложить на сумму площадей треугольников, образованных фиксированной точкой и каждым ребром.

Поскольку формула инвариантна относительно сдвига, можно считать, что одна вершина находится в начале координат $(0, 0)$ не умаляя общности.

Для треугольника с вершинами $(0, 0)$, (x_1, y_1), (x_2, y_2) площадь $S = \frac{1}{2}\,|x_1 y_2 - x_2 y_1|$.

Шнуровка для трёх точек: $x_0 y_1 - x_1 y_0 + x_1 y_2 - x_2 y_1 + x_2 y_0 - x_0 y_2 = 0 \cdot y_1 - x_1 \cdot 0 + x_1 \cdot y_2 - x_2 \cdot y_1 + x_2 \cdot 0 - 0 \cdot y_2 = x_1 y_2 - x_2 y_1$.

Таким образом, $S = \frac{1}{2}\,|x_1 y_2 - x_2 y_1|$, что совпадает.

Для многоугольников площадь аддитивна, поэтому формула верна.

```
double polygonArea(const vector<Point>& p) {
    double area = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        area += cross(p[i], p[j]);
    }
    return fabs(area) / 2;
}
```

Знак площади зависит от порядка обхода: положительный для против часовой стрелки, отрицательный для по часовой. Формула инвариантна относительно сдвига начала координат.

Центроид (центр масс)

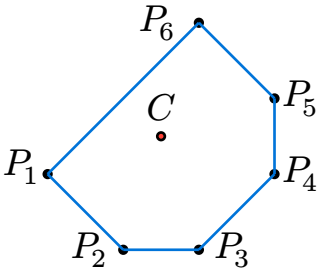
Центроид многоугольника — это геометрический центр или центр масс фигуры, точка, в которой фигура находится в равновесии.

Для многоугольника центроид (C_x, C_y):

$$C_x = \frac{1}{6S}\Sigma_{i=1}^{n}(x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

$$C_y = \frac{1}{6S}\Sigma_{i=1}^{n}(y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

Где S – площадь.



$$C_x = \frac{89}{6 \cdot 5.5} \approx 2.70$$

$$C_y = \frac{76}{6 \cdot 5.5} \approx 2.30$$

Доказательство

Центроид можно вычислить, разложив многоугольник на треугольники с вершиной в начале координат и взяв взвешенное среднее их центров.

Для треугольника с вершинами $(0,0)$, (x_1, y_1), (x_2, y_2) центроид находится в $\left(\frac{x_1 + x_2}{3}, \frac{y_1 + y_2}{3}\right)$.

Площадь треугольника $S_t = \frac{1}{2}\,|x_1 y_2 - x_2 y_1|$.

Тогда вклад в центроид: $\left(S_t \cdot \frac{x_1 + x_2}{3}, S_t \cdot \frac{y_1 + y_2}{3}\right)$.

Суммируя по всем треугольникам и деля на общую площадь, получаем формулу.

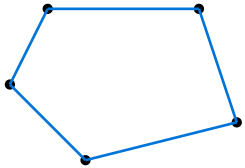
```
Point polygonCentroid(const vector<Point>& p) {
    double cx = 0, cy = 0, area = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        double cr = cross(p[i], p[j]);
        area += cr;
        cx += (p[i].x + p[j].x) * cr;
        cy += (p[i].y + p[j].y) * cr;
    }
    area /= 2;
    cx /= (6 * area);
    cy /= (6 * area);
    return Point(cx, cy);
}
```

Выпуклость

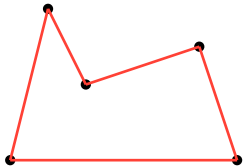
Многоугольник называется **выпуклым**, если он лежит по одну сторону от любой своей стороны, или, эквивалентно, если все его внутренние углы <= 180°.

Выпуклый многоугольник можно определить через ориентацию поворотов: все последовательные повороты (определяемые векторными произведениями) должны быть в одну сторону.

Выпуклый пятиугольник



Вогнутый пятиугольник



```
bool isConvex(const vector<Point>& p) {
    int n = p.size();
    if (n < 3) return false;
    bool sign = false;

    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        int k = (i + 2) % n;
        double cross = cross(p[j] - p[i], p[k] - p[j]);
        if (fabs(cross) > EPS) {
            if (sign == false) sign = cross > 0;
            else if (sign != (cross > 0)) return false;
        }
    }
    return true;
}
```

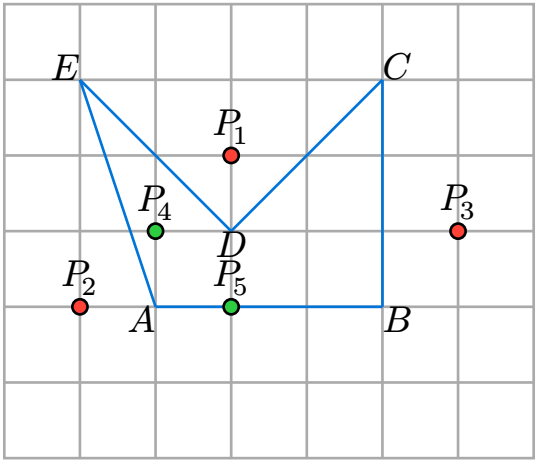
Проверка простоты многоугольника

Простой многоугольник не пересекает сам себя. Проверяется пересечением всех пар несмежных ребер.

```
bool isSimple(const vector<Point>& p) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 2; j < n; j++) {
            if (i == 0 && j == n - 1) continue;
            Segment s1 = {p[i], p[(i+1)%n]};
            Segment s2 = {p[j], p[(j+1)%n]};
            if (segmentsIntersect(s1, s2)) return false;
        }
    }
    return true;
}
```

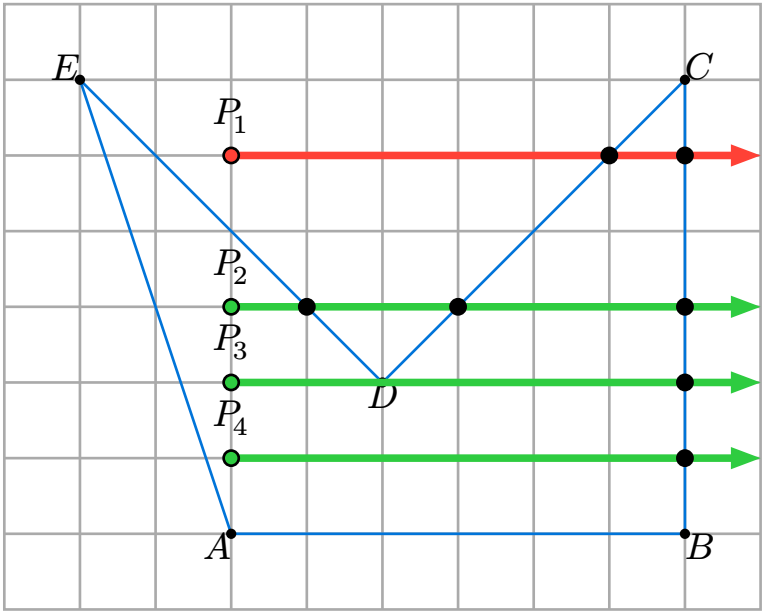
Точка внутри многоугольника

Ранее мы уже решали задачу о проверке принадлежности точки прямоугольнику. Посмотрим как решается это задача в общем виде.



Метод лучей (Ray Casting)

Провести луч из точки вправо, посчитать пересечения с ребрами. Нечетное число – внутри.



Следует игнорировать касания в вершинах, либо считать их дважды.

Код метода лучей (Ray Casting)

```
bool pointInPolygon(const Point& q, const vector<Point>& p) {
    int n = p.size();
    bool inside = false;
    for (int i = 0, j = n - 1; i < n; j = i++) {
        if ((p[i].y > q.y) != (p[j].y > q.y) &&
            (q.x < p[i].x+(p[j].x-p[i].x)*(q.y-p[i].y)/(p[j].y-p[i].y)))
            inside = !inside;
    }
    return inside;
}
```

Первое условие $(p[i].y > q.y) \neq (p[j].y > q.y)$ проверяет, что ребро от $p[j]$ к $p[i]$ пересекает горизонтальную линию на высоте $q.y$ (одна вершина выше, другая ниже).

Второе условие вычисляет x-координату точки пересечения ребра с этой горизонтальной линией и проверяет, что она правее $q.x$.

Если оба условия истинны, луч пересекает ребро, и флаг `inside` инвертируется.

Определение намотки (Winding Number)

Намотка (winding number) точки q относительно замкнутой кривой γ — это число полных оборотов, которое делает кривая вокруг точки, нормированное на 2π.

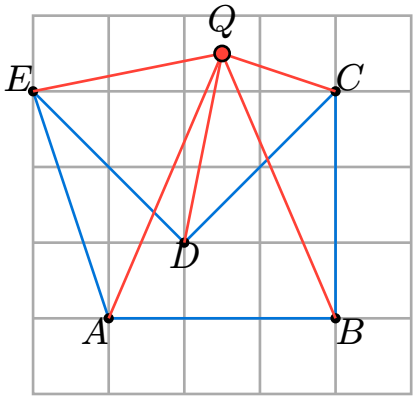
Математически: $w(q, \gamma) = \dfrac{1}{2\pi} \int_\gamma d\theta$, где $d\theta$ — дифференциал угла от вектора от q к точке на кривой.

Для многоугольника с вершинами $P_1, \ldots, P_n$ намотка вычисляется как сумма углов между векторами $\overrightarrow{P_i - q}$ и $\overrightarrow{P_{i+1} - q}$.

Метод ненулевой намотки (Winding Number)

Посчитать суммарный угол, на который “наматывается” многоугольник вокруг точки.

Если |w| >= 1 – внутри.



Для точки Q снаружи многоугольника сумма углов $w = \theta_1 + \theta_2 + ... = 0$.

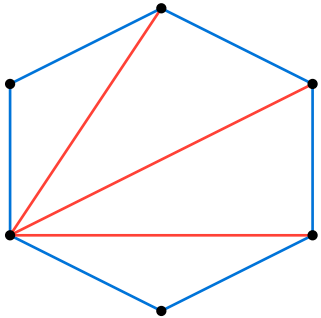
Для точки внутри $|w| \geq 1$.

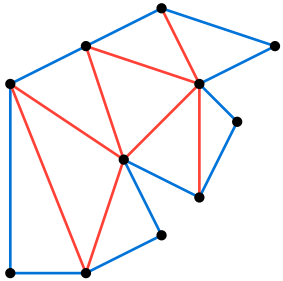
```
double windingNumber(const Point& q, const vector<Point>& p) {
    double wn = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        Point a = p[i] - q;
        Point b = p[(i + 1) % n] - q;
        double ang1 = angle(Point(0, 0), a);
        double ang2 = angle(Point(0, 0), b);
        double ang = ang2 - ang1;
        while (ang > PI) ang -= 2 * PI;
        while (ang < -PI) ang += 2 * PI;
        wn += ang;
    }
    return wn / (2 * PI);
}
```

Триангуляция многоугольника

Триангуляция многоугольника – это разбиение многоугольника на треугольники путём добавления непересекающихся диагоналей. Любая простой многоугольник на плоскости может быть триангулирована.

Для выпуклого многоугольника триангуляция тривиальна: все диагонали из одной вершины. Для вогнутого требуется более сложный алгоритм, такой как отсечение ушей (ear clipping).





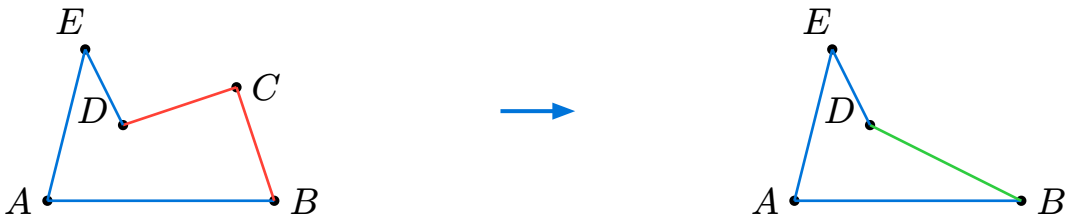
Алгоритм отсечения ушей (Ear Clipping)

Ухо – это вершина v_i, для которой треугольник v_{i-1}, v_i, v_{i+1} не содержит других вершин многоугольника и диагональ $v_{i-1}v_{i+1}$ лежит внутри многоугольника.

Алгоритм:

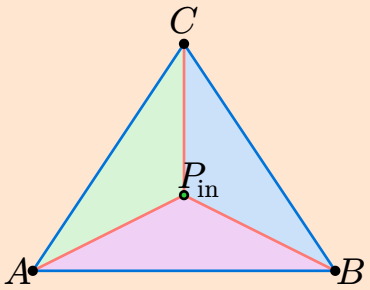
1. Построить список вершин.
2. Пока в списке больше 3 вершин:
   - Найти ухо.
   - Добавить диагональ из предыдущей и следующей вершин уха.
   - Удалить ухо из списка.

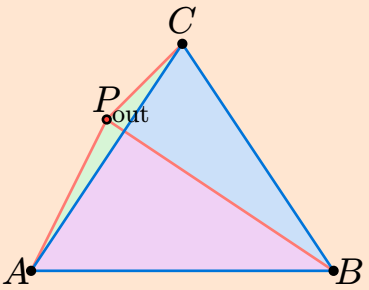
Работает за $O(n^2)$ в худшем случае, но часто быстрее.



```
bool pointInTriangle(Point q, Point a, Point b, Point c) {
    double area = fabs(cross(b - a, c - a));
    double area1 = fabs(cross(q - a, b - a));
    double area2 = fabs(cross(q - b, c - b));
    double area3 = fabs(cross(q - c, a - c));
    return fabs(area - (area1 + area2 + area3)) < EPS;
}
```

Для проверки принадлежности здесь считается сумма площадей треугольников образованных точкой и сторонами треугольника. Если точка внутри, то площадь совпадёт с площадью треугольника.





```
bool isEar(int i, const vector<Point>& p) {
    int n = p.size();
    int prev = (i - 1 + n) % n;
    int next = (i + 1) % n;
    for (int j = 0; j < n; j++) {
        if (j == i || j == prev || j == next) continue;
        if (pointInTriangle(p[j], p[prev], p[i], p[next])) return false;
    }
    return true;
}
```

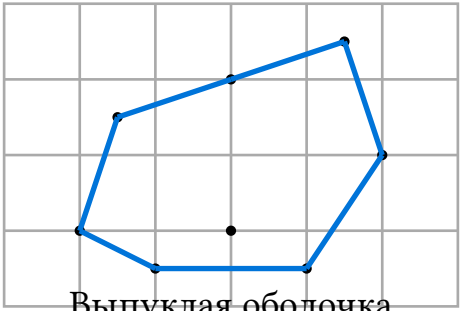
```
vector<pair<int,int>> triangulate(const vector<Point>& poly) {
    int n = poly.size();
    vector<pair<int,int>> diagonals;
    set<int> indices;
    for (int i = 0; i < n; i++) indices.insert(i);
    while (n > 3) {
        bool found = false;
        for (int i = 0; i < n; i++) {
            if (isEar(indices[i], poly)) {
                int prev = (i - 1 + n) % n;
                int next = (i + 1) % n;
                diagonals.push_back({indices[prev], indices[next]});
                indices.erase(indices.begin() + i);
                n--;
                found = true;
                break;
            }
        }
        assert(found);
    }
    return diagonals;
}
```

Выпуклая оболочка

Определение

Выпуклая оболочка множества точек S — это наименьший выпуклый многоугольник, содержащий все точки из S. Другими словами, это пересечение всех выпуклых множеств, содержащих S.

Множество называется **выпуклым**, если вместе с любыми двумя точками содержит и весь отрезок между ними.



Выпуклая оболочка

Свойства выпуклой оболочки

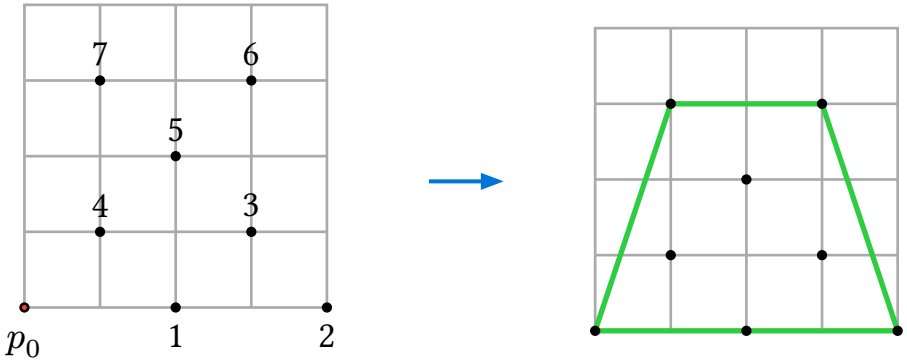
- Выпуклая оболочка всегда выпукла и содержит все исходные точки.
- Вершины оболочки — это точки из исходного множества.
- Выпуклая оболочка однозначно определена.
- Для n точек в общем положении оболочка имеет $O(n)$ вершин.

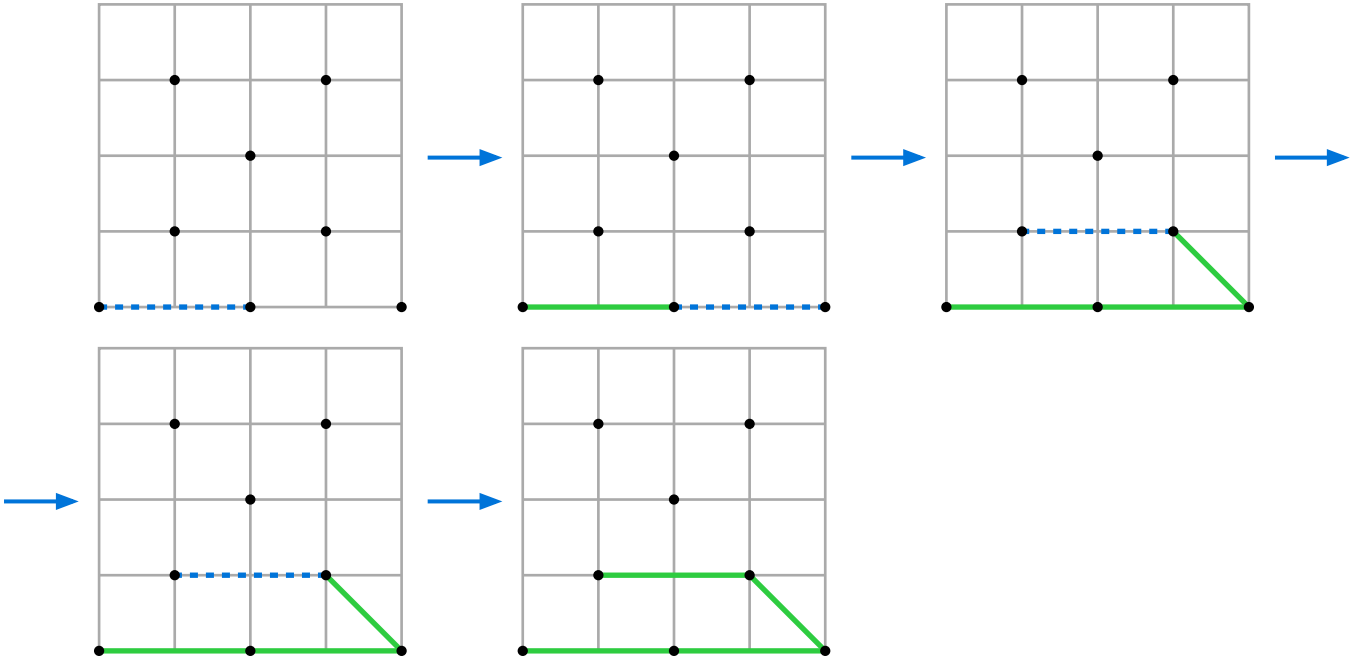
Алгоритм Грэхема

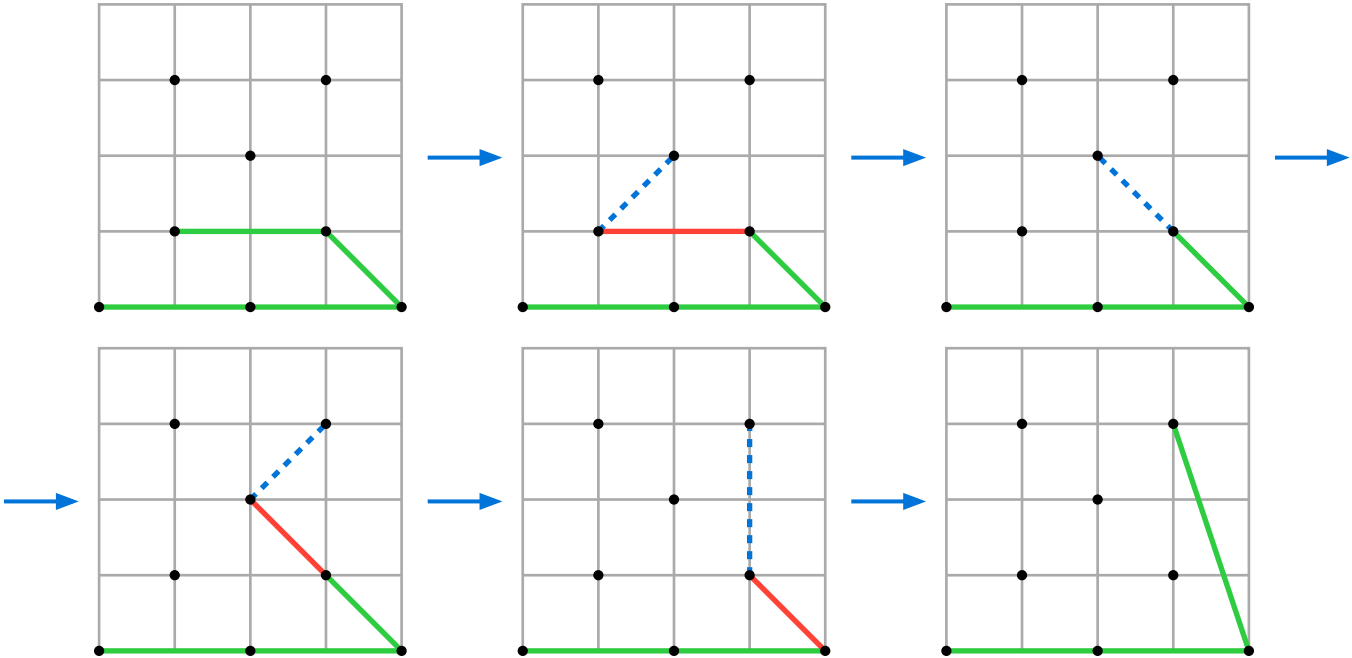
Алгоритм Грэхема строит выпуклую оболочку за $O(n \log n)$ времени.

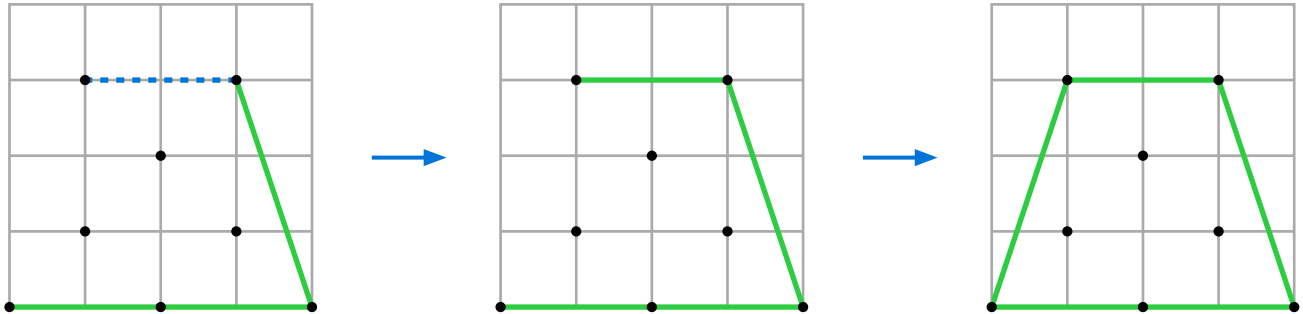
Шаги:

1. Выберем p_0 с минимальным y и x.
2. Отсортировать остальные точки по полярному углу относительно p_0. Если углы равны, ближе к p_0 идут первыми.
3. Итеративно строить оболочку: для каждой точки проверять, не нарушает ли она выпуклость, удаляя предыдущие точки при необходимости.









Выбор p_0

```
int findMinIndex(const vector<Point>& pts) {
    int n = pts.size();
    int minIdx = 0;
    for (int i = 1; i < n; i++) {
        if (pts[i].y < pts[minIdx].y || (pts[i].y == pts[minIdx].y &&
pts[i].x < pts[minIdx].x)) {
            minIdx = i;
        }
    }
    return minIdx;
}
```

Сортировка

```
void sortByPolarAngle(vector<Point>& pts, const Point& p0) {
    sort(pts.begin() + 1, pts.end(), [&](const Point& a, const Point& b) {
        double ang1 = angle(p0, a);
        double ang2 = angle(p0, b);
        if (fabs(ang1 - ang2) < EPS) return dist(p0, a) < dist(p0, b);
        return ang1 < ang2;
    });
}
```

Построение оболочки

```
vector<Point> buildHull(const vector<Point>& pts) {
    vector<Point> hull;
    for (auto& p : pts) {
        while (hull.size() >= 2 && orientation(hull[hull.size()-2],
hull.back(), p) != 2) {
            hull.pop_back();
        }
        hull.push_back(p);
    }
    return hull;
}
```

Основная функция

```
vector<Point> convexHull(vector<Point> pts) {
    int n = pts.size();
    if (n <= 1) return pts;

    int minIdx = findMinIndex(pts);
    swap(pts[0], pts[minIdx]);

    Point p0 = pts[0];
    sortByPolarAngle(pts, p0);

    return buildHull(pts);
}
```

Это не единственный способ нахождения минимальной Выпуклой оболочки, однако самый применяемый в рамках спортивного программирования. Если интересуют другие способы, то можно почитать про:

- Алгоритм Джарвиса (заворачивания подарка)
- Алгоритм Эндрю как модификация алгоритма джарвиса. Так же он применяется для триангуляции.